PROJECT PROPOSAL
JAVA CHECKERS GAME

A Checkers Game with Player-vs-Player and Player-vs-AI Modes

1. Introduction

The proposed project is a checkers (draughts) game developed in Java. The system is intended to provide a playable checkers environment supporting two main modes: Player-vs-Player (PvP) and Player-vs-AI (1vAI). The project focuses on representing an 8x8 board, handling legal moves and mandatory captures, managing turns and game states, and providing an interactive interface.

The project also provides an opportunity to apply object-oriented programming, data structures, game-state management, and basic artificial intelligence concepts in a single integrated software system.

2. Problem Statement

A checkers application is more than a graphical board that accepts piece movements. The system must represent regular pieces and kings, determine whether moves are legal, enforce rules such as mandatory captures and multi-jump sequences, maintain the current state of the game, and distinguish between player actions and AI decisions. In 1vAI mode, the system also needs a method for considering possible moves and selecting a move according to an evaluation approach.

The problem therefore involves the interaction of game rules, board state, user input, move processing, and AI decision-making within one application.

3. CCP Justification

The project can be considered a Complex Computing Problem (CCP) because several aspects require analysis and integration beyond a basic CRUD or interface-based application.

3.1 Depth of Knowledge Required

The project involves knowledge of object-oriented programming, data structures, game-state representation, move validation, and basic game AI. The 1vAI mode may involve concepts such as game trees, the minimax algorithm, and board evaluation. These concepts require more than simply creating forms and storing records.

3.2 Conflicting Requirements

The system contains requirements that can compete with each other. For example, a deeper AI search can improve the quality of move selection but can also increase processing time. Similarly, the interface should remain simple enough for players while still providing the information needed to understand and operate the game, such as highlighted legal moves and forced captures. The design therefore involves balancing AI decision quality, responsiveness, and usability.

3.3 Depth of Analysis Required

There is more than one possible approach to representing the board, validating moves, handling multi-jump captures, evaluating positions, and selecting AI moves. These alternatives can be considered using criteria such as correctness, simplicity, performance, maintainability, and suitability for the Java environment. The project therefore involves analysis rather than relying on a single obvious implementation.

3.4 Interdependence

The main components are dependent on one another. The board representation affects move validation; move validation affects game-state updates; the game state affects turn management and AI evaluation; and the AI's selected move must be processed by the same game rules used for player moves. Changes in one component can therefore affect the behavior of other components.

4. Project Objectives

- Develop a Java-based checkers application with an interactive board.
- Provide Player-vs-Player gameplay.
- Provide Player-vs-AI gameplay.
- Represent pieces, kings, board positions, turns, and relevant game states.
- Handle movement, capture rules, and rule validation through program logic.
- Explore an AI approach for selecting moves in 1vAI mode.
- Apply object-oriented programming and suitable data structures within the project.

5. Proposed System Features

- Interactive checkers board.
- Player-vs-Player mode.
- Player-vs-AI mode.
- Turn management.
- Piece movement and legal-move checking, including mandatory captures and multi-jumps.
- King promotion when a piece reaches the opposite end of the board.
- Game-state tracking and end conditions such as a player having no remaining pieces or no legal moves.
- AI move selection based on an evaluation/search approach.

The exact depth and implementation of individual features may depend on the project design and the requirements established during development.

6. Proposed System Architecture

A modular architecture can be used to separate the main responsibilities of the application. A possible structure consists of:

- **User Interface Layer –** displays the board and receives player actions.
- **Game Logic Layer –** manages turns, moves, captures, rules, and game state.
- **Board and Piece Model –** represents the checkers board, regular pieces, and kings.

- **AI Component –** considers available moves and evaluates positions for 1vAI mode.
- **Supporting Data Structures –** maintain information required for board state, moves, or game history.

These components are conceptually separated while remaining connected through the flow of a checkers move: player input or AI decision → move validation → board/game-state update → interface update.

7. Benchmarking with Existing Systems / Literature

Checkers software commonly provides a graphical board, legal move handling with forced captures, different game modes, and computer opponents. Existing checkers applications demonstrate that a complete system requires both accurate rule processing and some form of position evaluation or search for computer play.

The proposed project can be compared with existing checkers implementations using general criteria such as gameplay modes, move validation, board representation, AI decision-making, interface simplicity, and computational requirements. The purpose of the comparison is to understand common approaches rather than to reproduce a particular existing system.

8. Tools and Technologies

- Programming Language: Java
- Development Environment: VS Code
- User Interface: Java Swing or another suitable Java GUI approach
- Core Concepts: Object-Oriented Programming, Data Structures, Algorithms, Game-State Management
- AI Concepts: Minimax game-tree search and position evaluation concepts, as appropriate to the implementation

9. Expected System Structure

The application is expected to organize the checkers board, pieces, game rules, player controls, and AI logic into manageable components. The PvP mode can use player input from both sides, while the 1vAI mode can use player input for one side and an AI decision process for the other side.

The design is intended to keep the game logic sufficiently independent from the user interface so that rules and AI behavior can be considered separately from visual presentation.

10. Conclusion

The proposed Java Checkers Game combines an interactive application with algorithmic game logic and AI-based decision making. Its CCP characteristics arise particularly from the knowledge required to combine checkers rules with search and evaluation concepts, the trade-off between AI quality and processing requirements, the analysis of alternative implementation approaches, and the interdependence of the application's components. The project provides a suitable context for applying software construction, object-oriented programming, data structures, and algorithmic concepts in one system.